

Projekt: Cyfrowy bliźniak i testbench diagnostyczny ładowarek AC

Model-based diagnostyka, weryfikacja licznika i raporty do producenta

Dawid Cekała · zadanie rekrutacyjne AMPERE POINT — część 2 · wersja 2 · 15 czerwca 2026

Streszczenie (w jednym oddechu)

Buduję **cyfrowego bliźniaka** ładowarki AC w Simulink/Simscape: model „idealnej, zgodnej z normą” ładowarki, który służy jako **złoty wzorzec**. Do tego **emulator samochodu** ze scenariuszami i **wstrzykiwaniem usterek**, który prowadzi zarówno wzorzec, jak i (opcjonalnie, przez tani adapter) realną ładowarkę. **Komparator** porównuje zachowania i wystawia werdykt pass/fail oraz **twardy, powtarzalny raport do producenta w Chinach**. Całość projektuję i demonstruję w symulacji — bez kapitału, w ok. 40 h. To zamienia obecny, podstawowy „tester wallboxów”, który firma rozdaje w gratisie, w realne narzędzie diagnostyki i kontroli jakości, dopasowane do modelu „OEM na sterydach”.

1. Najpierw zrozummy ładowarkę AC (od zera)

To moje pierwsze zetknięcie z tymi ładowarkami, więc najpierw układałam fundament — bez tego koncepcja nie ma sensu.

1.1. Ładowarka AC to „inteligentny włącznik”, nie prostownik

Bateria auta przyjmuje prąd stały (DC). Z gniazda płynie prąd przemienny (AC). Zamiana AC→DC (prostownik) siedzi **w samochodzie** (ładowarka pokładowa). Wallbox czy ładowarka przenośna **nie zamieniają prądu** — one tylko bezpiecznie podają AC i **dogadują się z autem**, ile prądu może popłynąć. Wniosek dla projektu: cała „inteligencja” ładowarki AC to **sygnalizacja, sterowanie i zabezpieczenia** — i to właśnie da się modelować i testować od zewnątrz, bez zaglądania do środka.

1.2. Rozmowa ładowarki z autem — Control Pilot (handshake)

Komunikacja idzie jedną żyłą sygnałową — **Control Pilot (CP)**. Ładowarka podaje na nią napięcie, a auto swoimi rezystorami „ściąga” je do konkretnych poziomów; każdy poziom to inny **stan**. Gdy auto jest gotowe, ładowarka nadaje na CP **sygnał PWM** (prostokątny, 1 kHz), którego **wypełnienie** koduje maksymalny prąd. To jest serce normy **IEC 61851** i serce mojego modelu.

Stan	Napięcie CP	Znaczenie
A	+12 V	brak auta (spoczynek)
B	+9 V	auto podłączone, jeszcze nie ładuje
C	+6 V	auto ładuje
D	+3 V	ładowanie z wymuszoną wentylacją (rzadkie)
E / F	0 / -12 V	błąd / ładowarka niedostępna

Kodowanie prądu: **prąd [A] = wypełnienie [%] × 0,6** (w zakresie 10–85%). Np. 25% → 15 A, 50% → 30 A, ok. 53% → 32 A. Druga żyła, **Proximity Pilot (PP)**, mówi rezystorem, jaki kabel jest podłączony i ile prądu wytrzyma.

1.3. Licznik energii — i dlaczego jego dokładność jest krytyczna

Ładowarki AMPERE POINT mają **niezerowalny licznik energii (kWh)**. Ze strony firmy: to filar oferty dla flot — pracownik ładuje auto służbowe w domu, a firma rozlicza koszt na podstawie odczytu licznika (seria Q dodaje historię i eksport w aplikacji Tuya). **Jeśli licznik myli się o kilka procent, klient dostaje złe faktury** — a to spór i utrata zaufania. Dokładność licznika to więc nie detal, tylko realna wartość biznesowa, którą da się i warto testować.

1.4. DLB — równoważenie mocy (gdzie najczęściej coś „nie gra”)

Wallbox PRIME ma **Dynamic Load Balancing (DLB)** do 4 stacji: rozdziela dostępną moc tak, by nie wybić bezpiecznika. To klasyczne miejsce na błędy: ładowarka musi **w porę** (norma: w kilka sekund) zmniejszyć prąd, gdy w domu rośnie inne obciążenie, i ponownie go zwiększyć, gdy auto się odłączy. Wolna albo błędna reakcja = wybity bezpiecznik lub niepotrzebnie wolne ładowanie.

1.5. Mapa awarii — gdzie realnie powstają problemy

- **Handshake CP:** zły poziom stanu, brak diody (auto „udaje” kabel), nieczyste przejścia B→C.
- **Prąd:** ładowarka oferuje złe wypełnienie PWM → za wolno albo ponad bezpiecznik.
- **Reakcja w czasie:** zbyt wolna odpowiedź na polecenie zmniejszenia prądu (DLB).
- **Licznik:** przekłamanie kWh → złe rozliczenia floty.
- **Bezpieczeństwo:** brak reakcji na usterkę (zwarcie CP, brak diody) zamiast odcięcia.
- **Regresje firmware:** nowy firmware od producenta „naprawia jedno, psuje drugie”.

2. Realia AMPERE POINT — i co z nich wynika

Zebrane z ich strony i z rozmowy — to fundament dopasowania projektu:

- **„OEM na sterydach”.** Zamawiają gotowy sprzęt (z Azji), magazynują w Polsce (Tuchom k. Gdańska, wysyłka 48 h), testują, wdrażają i sprzedają bezpośrednio. Hasło ze strony: **„każda jednostka przechodzi wiele testów”**.
- **Zamknięty firmware.** Kod jest producenta (Chiny), nie open source. Firma **nie poprawia kodu** — zgłasza uwagi, a producent przysyła zmodyfikowane elementy lub nowy firmware. Czyli ładowarka to dla nich **czarna skrzynka**.
- **Świadomie BEZ OCPP.** Strona wprost: „bez drogich wdrożeń, bez OCPP, bez skomplikowanych dashboardów”. Stawiają na prosty sprzęt + licznik energii do rozliczeń. Mój projekt **nie może** opierać się na OCPP — działa na warstwie fizycznej (CP/PP), mocy i licznika.
- **Strategia małych prototypów.** Testują niewiele sztuk i szybko adaptują się do klienta, żeby nie topić kapitału. Potrzebują narzędzi **tanich, szybkich i powtarzalnych**.
- **Już rozdają „tester wallboxów” w gratisie.** To podstawowy gadżet (lampki: jest/nie ma napięcia, stan). **Mój projekt to świadome rozwinięcie tego pomysłu** z „świeci się / nie świeci” do realnej diagnostyki i kontroli jakości.

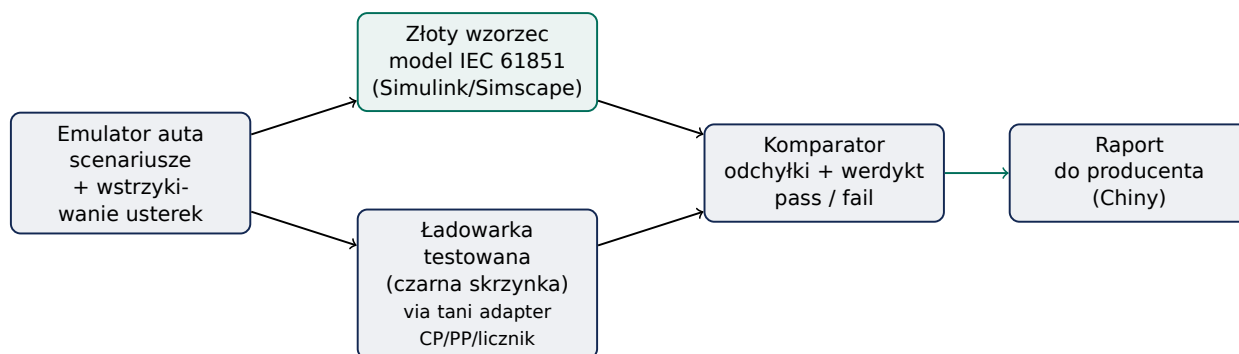
Wniosek: firma potrzebuje taniego, powtarzalnego, automatycznego narzędzia, które (a) charakteryzuje czarną skrzynkę od zewnątrz, (b) łapie regresje po nowym firmware, (c) weryfikuje licznik energii, (d) zamienia mgliste „nie działa” w **twardy raport dla producenta**. Dokładnie to projektuję.

3. Koncepcja: cyfrowy bliźniak + testbench diagnostyczny

Pomysł w jednym zdaniu: skoro nie mam kodu ładowarki, to opisuję jej **poprawne zachowanie** jako model (złoty wzorzec) i sprawdzam, gdzie realna ładowarka od niego odbiega — automatycznie i powtarzalnie.

Cztery komponenty:

1. **Złoty wzorzec (model referencyjny).** Model w Simulink/Stateflow + Simscape, który zachowuje się jak ładowarka **idealnie zgodna z IEC 61851**: stany CP, wypełnienie PWM → prąd, czasy reakcji, logika DLB, licznik. To „prawda”, do której porównujemy.
2. **Emulator auta + scenariusze + wstrzykiwanie usterek.** Druga maszyna stanów udaje samochód i prowadzi test: normalne ładowanie, przypadki brzegowe i celowe usterki (brak diody, złe PP, nagłe zmiany, wolna reakcja, skok obciążenia w DLB).
3. **Komparator + werdykt + raport.** Porównuje odpowiedź wzorca i obiektu testowanego, liczy odchyłki (prąd, czas reakcji, błąd licznika) i wystawia **pass/fail** oraz gotowy **raport do producenta** (co, kiedy, o ile, jak odtworzyć).
4. **Opcjonalny tani adapter sprzętowy.** Mały front-end CP/PP + pomiar, który podpiną **realną** ładowarkę do tej samej logiki testowej. Najpierw wszystko projektuję i waliduję w symulacji — sprzęt to cienki most na później.



Czym to się różni od ich obecnego „testera wallboxów”

Obecny gadżet odpowiada na pytanie „**czy w ogóle działa?**” (go/no-go). Mój testbench odpowiada na „**jak dokładnie się zachowuje, gdzie odbiega od normy, o ile, i czy nowy firmware tego nie zepsuł?**” — czyli charakteryzacja, regresja i materiał dowodowy dla producenta.

4. Co dokładnie symuluję (Simulink/Simscape) — i wolne alternatywy

Rdzeń modelu

- **Simscape Electrical — analogowy front-end CP.** Źródło ± 12 V/PWM, sieć rezystorów auta ($2,74\text{ k}\Omega$ + dioda, dołączany $1,3\text{ k}\Omega$), dzielnik. Widzę realne poziomy $12/9/6\text{ V}$ i kształt PWM — dowód, że rozumiem warstwę fizyczną, nie tylko logikę.
- **Stateflow — maszyna stanów CP.** Dwie sztuki: złoty wzorzec (ładowarka) i emulator auta. Pełen handshake $A \rightarrow B \rightarrow C$ i z powrotem, z czasami przejść.
- **Logika prąd/PP.** Przeliczenie wypełnienie $\rightarrow$ prąd ($\times 0,6$), kodowanie PP, wartości „raportowane” jak w aplikacji.

Scenariusz A (kluczowy): weryfikacja licznika energii

Zadaję znany profil prądu w sesji, liczę **prawdziwą energię** (całka mocy po czasie), a model licznika obarczam konfigurowalnym błędem (offset, dryf, kwantyzacja). Testbench wykrywa, gdy **raportowane kWh** odbiega ponad tolerancję (np. $\pm 1\%$). Efekt: zanim sprzęt trafi do floty, wiem, że licznik nie zawyża faktur — a jeśli zawyża, mam dokładną liczbę do producenta.

Scenariusz B (pokaz integracji): DLB

Modeluję dom: bezpiecznik główny + zmienne obciążenie domowe + 1–4 ładowarki z DLB. Podnoszę obciążenie i sprawdzam, czy suma prądów **nigdy nie przekracza** bezpiecznika i czy ładowarka zdążyła zmniejszyć prąd w wymaganym czasie. Odpinam auto i sprawdzam, czy pozostałe „dobierają” zwolnioną moc. To wprost dotyka realnych skarg: wybity bezpiecznik / wolne ładowanie.

Wolne alternatywy (gdyby brak MATLAB-a)

Python (NumPy/SciPy + prosta maszyna stanów, np. transitions) do logiki i scenariuszy; **Scilab/Xcos** jako darmowy odpowiednik Simulinka; **LTspice / ngspice / Falstad** do analogowego obwodu CP. Koncepcja jest niezależna od narzędzia — MATLAB (licencja uczelniana) tylko ją przyspiesza.

5. Plan pracy na ok. 40 h

Faza	Zakres	h
0	Research normy IEC 61851 (CP/PP, czasy) + lista przypadków testowych	4
1	Simscape: analogowy front-end CP (poziomy 12/9/6 V, PWM)	6
2	Stateflow: maszyna stanów — złoty wzorzec + emulator auta	8
3	Logika wypełnienie → prąd + kodowanie PP + wartości raportowane	3
4	Wstrzykiwanie usterek + komparator + werdykt pass/fail	7
5	Scenariusz A: weryfikacja licznika energii (błąd kWh)	5
6	Scenariusz B: DLB (bezpiecznik + obciążenia + 2–4 ładowarki) — elastyczny	4
7	Generator raportu do producenta + demo + opis koncepcji	5
Razem		ok. 42

Twardy rdzeń (fazy 0–5, 7) to ok. 38 h — mieści się w budżecie nawet bez fazy DLB. Faza 6 jest „buforem jakości”: robię, jeśli rdzeń idzie sprawnie.

6. Co to realnie wyłapie (wartość diagnostyczna)

Każdy przypadek to gotowy, odtwarzalny wpis do raportu dla producenta:

- **Licznik zawyża/zaniża kWh** ponad tolerancję → złe rozliczenia floty (uderza w ich główny argument B2B).
- **Złe wypełnienie PWM** → ładowarka oferuje inny prąd, niż powinna (za wolno albo ponad bezpiecznik).
- **Brak reakcji na usterkę** (brak diody, zwarcie CP) zamiast odcięcia → problem bezpieczeństwa.
- **Za wolna reakcja DLB** (> wymóg normy) → ryzyko wybicia bezpiecznika u klienta.

- **Regresja po nowym firmware:** przypadek, który wcześniej przechodził, teraz nie → złapane **przed** wysyłką do klientów.

Zamiast „klient mówi, że coś nie działa” producent dostaje: **który scenariusz, jaka odchyłka, o ile, jak odtworzyć**. To realnie skraca pętlę poprawek z chińską fabryką.

7. Budżet — kapitał bliski zera

Pozycja	Koszt
Symulacja: MATLAB/Simulink/Simscape (licencja uczelniana)	0 zł
albo wariant darmowy: Python + Scilab/Xcos + LTspice	0 zł
Czas: ok. 40 h pracy własnej	wkład własny
<i>Opcjonalnie — dopiero do testów realnej ładowarki:</i>	
Tani adapter CP/PP + pomiar (MCU + analog)	300-500 zł
Ładowarka do testów	sprzęt firmy
Koncept + symulacja (deliverable na poniedziałek)	ok. 0 zł kapitału

To wprost odpowiada na ich strategię „nie topić kapitału”: najpierw tani model, dopiero potem ewentualny sprzęt.

8. Ryzyka i ograniczenia (uczciwie)

- **Model jest tak dobry, jak norma, którą koduję** — dlatego faza 0 (rzetelne wczytanie IEC 61851) jest krytyczna.
- **Czarna skrzynka** — niektóre zachowania wnioskuje z zewnątrz; testbench wskazuje *co* odbiega, nie zawsze *dlaczego* (to już rola producenta).
- **Adapter dotyka napięcia sieci** — dlatego wersja symulacyjna jest pełnoprawnym deliverable, a sprzęt to osobny krok z zabezpieczeniami.
- **To nie certyfikacja** — narzędzie wspiera diagnostykę i QA, nie zastępuje akredytowanych badań bezpieczeństwa.

9. Dlaczego to pasuje do firmy (pitch)

AMPERE POINT zarabia na tym, że **lepiej testuje i szybciej adaptuje** cudzy sprzęt niż konkurencja — „OEM na sterydach”. Mój projekt zamienia ich pętlę „testuj → zbierz feedback → zgłoś producentowi” w **powtarzalny, tani, mierzalny proces**: złoty wzorzec łapie odchyłki, regresja pilnuje nowych firmware’ów, a weryfikacja licznika chroni ich główny argument dla flot. Zero kapitału na start, pełna symulacja jako dowód koncepcji — i jasna ścieżka do wersji sprzętowej, gdy się opłaci.